

TP1 - Détection d'objets avec YOLOv11n et OpenVINO

Corrigé

Table des matières

1	Bases à avoir	2
2	Introduction	2
3	Partie 1 : Configuration du GPU Intel	2
3.1	Connexion et ouverture du projet	2
3.2	Configuration de l'environnement GPU	2
3.2.1	Gestion des permissions Linux	3
3.2.2	Le groupe render	3
3.3	Étapes de configuration	4
3.3.1	Première étape : Modification du Dockerfile	4
3.3.2	Deuxième étape : Ajout de l'utilisateur au groupe	4
3.3.3	Troisième étape : Modification du build_and_run.sh (docker_run)	4
3.3.4	Quatrième étape : Modification du build_and_run.sh (docker_build)	5
3.4	Installation des drivers et bibliothèques	5
3.4.1	Les drivers pour le GPU	6
3.4.2	Installation des bibliothèques pip, apt et python	7
3.4.3	Modification du Dockerfile pour les bibliothèques	7
3.4.4	Modifications finales du build_and_run.sh	9
3.5	Build et lancement du Docker	10
3.6	Installation des bibliothèques Python	10
3.7	Vérification du GPU	11
4	Partie 2 : Environnement ROS2	11
4.1	Création du package	11
4.2	Export du modèle YOLO en format OpenVINO	12
4.3	Création du code de détection	12
4.4	Modification du setup.py	12
4.5	Modification du package.xml	12
4.6	Compilation et lancement	12
5	Annexe : Problèmes et solutions	13
6	Liste des commandes utiles	15

1 Bases à avoir

- Bases sur les dockers
- Bases de ROS2 et de son fonctionnement
- Savoir approximativement comment fonctionne le robot (VS Code, Hôte/Docker)
- Connaître l'existence et savoir un peu utiliser Teltonika et noVNC

2 Introduction

Dans ce TP, le but va être de faire de la détection d'objets grâce à YOLOv11n. Le robot est équipé du CPU Intel Ultra 7 255H. Il possède une unité graphique intégrée. Notre but va être de l'utiliser. Grâce à elle, on va pouvoir décupler les performances de YOLO. Or le Docker n'a pas accès par défaut à cette unité graphique. Dans un environnement Docker, par sécurité, le conteneur est isolé du matériel de l'hôte. Pour qu'il puisse « toucher » au GPU Intel, il faut lui ouvrir une porte.

La première partie du TP va donc être de permettre au Docker d'y avoir accès. Pour pouvoir mieux gérer les calculs, on va utiliser OpenVINO qui est un toolkit développé par Intel spécialement pour les calculs GPU :

OpenVINO (Open Visual Inference and Neural network Optimization) est un toolkit gratuit développé par Intel. Son but est simple : faire en sorte que tes modèles d'IA (comme YOLO) tournent le plus vite possible sur du matériel Intel (ton processeur Ultra 7, ton GPU intégré Arc, et ton NPU). C'est un peu comme un « super-traducteur » et un « optimiseur » de code pour l'IA.

Dans un deuxième temps, il faudra mettre en place l'environnement de travail ROS2 (création des packages, ajout des codes, importation du modèle, etc.)

Le rendu final se fera sur noVNC où on aura un retour vidéo de la caméra et de la détection d'objets.

Une annexe est fournie à la fin et elle repertorie toutes les erreurs possibles et leurs solutions. Les codes sont aussi fournis.

3 Partie 1 : Configuration du GPU Intel

3.1 Connexion et ouverture du projet

1. Connecter le robot en SSH à VS Code
2. Ouvrir VS Code à la racine du projet donc à `roskit_ws`

3.2 Configuration de l'environnement GPU

On va configurer l'environnement pour avoir accès au GPU intégré du CPU Intel :

3.2.1 Gestion des permissions Linux

Dans le Docker, Linux gère les permissions avec un système d'utilisateurs et de groupes. Sous ROS 2, les « ressources » sont les composants concrets de ton robot : les capteurs (caméra dans `/dev/video0`, LiDAR en USB), les fichiers de configuration (fichiers `.yaml`), et les interfaces réseau pour faire passer tes messages (Topics/Services).

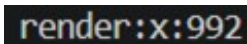
Pour que ton conteneur Docker puisse piloter le robot, Linux vérifie que l'utilisateur interne possède le bon « badge » : par exemple, être dans le groupe `video` pour voir l'image ou `dialout` pour lire le port série d'un capteur. Si ces permissions ne sont pas alignées entre ton PC et Docker, ton nœud ROS 2 se lancera mais ne pourra ni « voir » ni « entendre » le matériel, bloqué par une erreur de type *Permission denied*.

3.2.2 Le groupe render

Le groupe `render` est le groupe du GPU Intel, c'est donc celui-là qu'il faut utiliser. Or le Docker n'a pas accès à ce groupe, il n'a pas les permissions et n'est pas défini dans les fichiers du Docker. Il faudra donc l'ajouter manuellement plus tard.

Vérification du groupe render (hors du docker) :

```
cat /etc/group
```



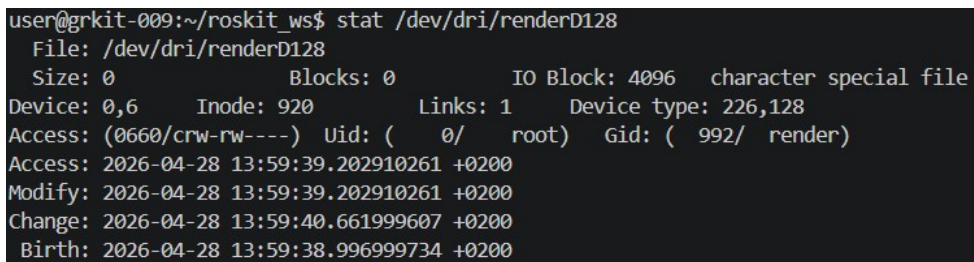
```
render:x:992
```

FIGURE 1 – ID du groupe render du CPU

On peut trouver l'ID du groupe render du CPU. (hors du docker)

```
stat /dev/dri/renderD128
```

On peut voir plus en détail ce groupe :



```
user@grkit-009:~/roskit_ws$ stat /dev/dri/renderD128
  File: /dev/dri/renderD128
  Size: 0          Blocks: 0          IO Block: 4096   character special file
Device: 0,6      Inode: 920          Links: 1        Device type: 226,128
Access: (0660/crw-rw----)  Uid: (  0/   root)   Gid: ( 992/  render)
Access: 2026-04-28 13:59:39.202910261 +0200
Modify: 2026-04-28 13:59:39.202910261 +0200
Change: 2026-04-28 13:59:40.661999607 +0200
 Birth: 2026-04-28 13:59:38.996999734 +0200
```

FIGURE 2 – Détails du groupe render

En faisant cette commande, on voit que le render est sur le groupe 992 : lors de l'installation des drivers sur l'hôte Ubuntu 24.04 (noble), Linux a créé ce groupe automatiquement et lui a assigné le GID (Group ID) 992. Ce numéro est attribué dynamiquement — sur une autre machine il pourrait être différent.

Dans le conteneur Ubuntu 22.04 (jammy), le groupe `render` existe aussi, mais il a été créé par Ubuntu avec le GID 109 par défaut — parce que jammy et noble assignent les GIDs différemment, de manière totalement indépendante.

Donc il y a un problème de GID qu'il faut corriger pour donner l'accès.

3.3 Étapes de configuration

3.3.1 Première étape : Modification du Dockerfile

Ajouter ces deux lignes dans le Dockerfile (dans `/roskit_ws/src/gr_docker`, c'est là que la configuration du Docker se fait) avant les dépendances :

```
ARG RENDER_GID=#your GID
RUN groupmod -g ${RENDER_GID} render || groupadd -g ${RENDER_GID} render
```

Elles permettent de synchroniser le GID avec l'hôte.

```
# Synchronise le GID render avec l'hôte
ARG RENDER_GID=992
RUN groupmod -g ${RENDER_GID} render || groupadd -g ${RENDER_GID} render

##### ADD NEW DEPENDENCIES JUST ABOVE THIS LINE #####
```

FIGURE 3 – Modification du Dockerfile

3.3.2 Deuxième étape : Ajout de l'utilisateur au groupe

Ajouter la ligne, toujours dans le Dockerfile :

```
RUN usermod -a -G render $USER
```

Avec cette ligne, on ajoute l'utilisateur au groupe `render`.

```
RUN useradd -ms /bin/bash -u $UID -g $GID $USER
RUN adduser $USER dialout
RUN addgroup $USER dialout
RUN adduser $USER video
RUN usermod -a -G video $USER
RUN usermod -a -G render $USER
```

FIGURE 4 – Modification du Dockerfile

On peut maintenant de nouveau vérifier le groupe `render` :

```
cat /etc/group
```

```
render:x:992:user
```

FIGURE 5 – Groupe render : ajout de l'utilisateur

Le `:user` indique que nous y avons maintenant accès. (une fois que tout aura été mis en place et build).

3.3.3 Troisième étape : Modification du `build_and_run.sh` (`docker_run`)

Maintenant on va dans le `build_and_run.sh` et on ajoute cette ligne dans le `docker_run` :

```
--group-add #your GID \
```

Elle permet au Docker de reconnaître le groupe **render** sur votre GID.

```
docker run \
  --net=host \
  --ipc=host \
  --pid=host \
  --restart=always \
  --init \
  --ulimit core=-1 \
  --privileged \
  -v /tmp/.X11-unix:/tmp/.X11-unix \
  -e DISPLAY=unix$DISPLAY \
  -v /dev:/dev \
  --group-add 992 \
  --env-file ./config/env_var.config \
  -it \
  --name $CONTAINER_NAME \
```

FIGURE 6 – Ajout du groupe dans le build_and_run.sh

3.3.4 Quatrième étape : Modification du build_and_run.sh (docker_build)

Maintenant toujours dans le `build_and_run.sh` mais dans le `docker_build`, on ajoute :

```
--build-arg RENDER_GID=$(getent group render | cut -d: -f3)
```

Cette ligne permet de lire le fichier `/etc/group` de l'hôte et extrait le troisième champ (le GID) de la ligne du groupe `render`. Donc on utilise le GID de l'hôte (votre GID) automatiquement.

```
docker build -f Dockerfile -t $IMAGE_NAME . \
  --build-arg RENDER_GID=$(getent group render | cut -d: -f3) \
  --build-arg USER=$USER \
  --build-arg GROUP=$GROUP \
  --build-arg UID=$USER_UID \
  --build-arg GID=$USER_GID \
  --build-arg ABSOLUTE_WORKSPACE=$ABSOLUTE_WORKSPACE
```

FIGURE 7 – Ajout du groupe dans le build and run.sh

Maintenant l'accès GPU à été mis en place dans les fichiers du docker.

3.4 Installation des drivers et bibliothèques

On va maintenant installer les drivers Intel pour le GPU et les bibliothèques nécessaires pour la détection d'objets. Puisqu'on modifie le `build_and_run.sh`, on doit le relancer à chacune de celles-ci. Or si on installe des bibliothèques directement dans le Docker, lorsqu'on relance le `.sh`, les bibliothèques disparaissent complètement puisqu'elles sont stockées dans le cache du Docker. Donc on va créer des dossiers externes au Docker dans lesquels les bibliothèques seront installées et on appellera ces bibliothèques depuis le Docker.

3.4.1 Les drivers pour le GPU

L'hôte est Ubuntu 24.04 (noble). Le repo Intel direct pour noble contient `libigc2` qui entre en conflit avec `libigc1` déjà installé par Ubuntu. Le PPA `kobuk-team` est maintenu par Canonical spécifiquement pour résoudre ce problème sur noble.

Explication PPA : Le PPA `kobuk-team/intel-graphics` est un dépôt de logiciels tiers. Le groupe `kobuk-team` maintient ce dépôt pour fournir des versions très récentes des pilotes graphiques Intel et des bibliothèques associées. En général, les dépôts officiels d'Ubuntu sont conservateurs : ils privilégient la stabilité et ne mettent pas à jour les pilotes vers les dernières versions disponibles. Ce PPA permet de contourner cette limitation pour améliorer les performances dans les jeux ou les applications 3D, bénéficier des dernières optimisations pour le décodage matériel (VA-API) ou supporter de nouveaux matériels Intel (processeurs Core de 12e, 13e, 14e génération ou cartes graphiques Arc) qui pourraient mal fonctionner avec les anciens pilotes (comme c'est le cas ici pour nous).

On peut donc rentrer ces commandes dans le terminal de l'hôte (donc hors du docker) :

```
# Ajoute le PPA Canonical pour Intel GPU
sudo add-apt-repository -y ppa:kobuk-team/intel-graphics
sudo apt update

# Installe les libs
sudo apt install -y \
    libigdgmm12 \
    libigc2 \
    libigdfcl2 \
    intel-opencl-icd \
    libze1 \
    libze-intel-gpu1 \
    clinfo

# Ajoute ton user aux groupes
sudo usermod -aG render $USER
sudo usermod -aG video $USER

# Applique les groupes dans la session courante sans reboot
newgrp render
```

En faisant la commande :

```
clinfo | grep Device Name
```

On voit les différents composants détectés (si plusieurs fois = normal car une fois par sous-device). `clinfo` est un outil de diagnostic pour OpenCL. OpenCL (Open Computing Language) est une interface standard qui permet aux programmes d'utiliser le GPU, CPU, ou d'autres accélérateurs pour faire des calculs parallèles (comme l'IA, le traitement d'images, etc.).

```
user@grkit-009:~/roskit_ws$ clinfo | grep "Device Name"
Device Name                                Intel(R) Graphics
Device Name                                Intel(R) Graphics
Device Name                                Intel(R) Graphics
Device Name                                Intel(R) Graphics
```

FIGURE 8 – GPU détecté par OpenCL

3.4.2 Installation des bibliothèques pip, apt et python

Il faut rentrer ces commandes dans le terminal de l'hôte :

```
# pip-cache : stocke les .whl t l charg s
mkdir -p $HOME/ros2-libs/pip-cache

# apt-cache : stocke les .deb t l charg s
mkdir -p $HOME/ros2-libs/apt-cache

# python-packages : les packages pip install s eux-m mes
mkdir -p $HOME/ros2-libs/python-packages
```

Ces commandes créées des dossiers dans lesquels on mettra les caches des bibliothèques installées.

3.4.3 Modification du Dockerfile pour les bibliothèques

Problèmes rencontrés et solutions apportées :

- **Problème 1** : Le repo Intel jammy contient deux versions incompatibles des libs Intel Compiler : `libigc1` (ancienne) et `libigc2` (nouvelle). Certains paquets comme `libze-intel-gpu1` demandent `libigc2 >= 2.11.7`, mais APT essayait d'installer `libigc1` en même temps → conflit.

Solution : épingler toutes les versions sur le build 1146 (le plus récent et cohérent) :

```
libigc2=2.11.12-1146~22.04
libigdfcl2=2.11.12-1146~22.04
intel-opencl-icd=25.18.33578.15-1146~22.04
```

Le numéro 1146 est le numéro de release interne d'Intel — tous les paquets du même build sont garantis compatibles entre eux.

- **Problème 2** : `python3-catkin-pkg` et `pip` n'existent pas sous ces noms dans jammy.

Solution : `pip` → `python3-pip`, `python3-catkin-pkg` → `python3-catkin-pkg-modules`.

- **Problème 3** : Après le bloc Intel qui fait `rm -rf /var/lib/apt/lists/*`, les étapes suivantes ne trouvaient plus les paquets Ubuntu car les listes étaient effacées.

Solution : ajouter `apt-get update` au début de chaque RUN qui installe des paquets après ce bloc.

Maintenant qu'on sait comment régler les problèmes, on va dans le Dockerfile et on ajoute :

```
# Drivers Intel GPU/NPU
RUN apt-get update && apt-get install -y curl gpg && \
```



```

curl -fsSL https://repositories.intel.com/gpu/intel-graphics.key | \
gpg --dearmor -o /usr/share/keyrings/intel-graphics.gpg && \
echo deb [arch=amd64 signed-by=/usr/share/keyrings/intel-graphics.gpg] \
https://repositories.intel.com/gpu/ubuntu jammy unified \
> /etc/apt/sources.list.d/intel-gpu-jammy.list && \
apt-get update && apt-get install -y \
libigdgmm12 \
libigc2=2.11.12-1146~22.04 \
libigdfcl2=2.11.12-1146~22.04 \
intel-opencl-icd=25.18.33578.15-1146~22.04 \
libze-intel-gpu1 \
libze1 \
clinfo \
&& rm -rf /var/lib/apt/lists/*

```

```

# Drivers Intel GPU/NPU
RUN apt-get update && apt-get install -y curl gpg && \
curl -fsSL https://repositories.intel.com/gpu/intel-graphics.key | \
gpg --dearmor -o /usr/share/keyrings/intel-graphics.gpg && \
echo "deb [arch=amd64 signed-by=/usr/share/keyrings/intel-graphics.gpg] \
https://repositories.intel.com/gpu/ubuntu jammy unified" \
> /etc/apt/sources.list.d/intel-gpu-jammy.list && \
apt-get update && apt-get install -y \
libigdgmm12 \
libigc2=2.11.12-1146~22.04 \
libigdfcl2=2.11.12-1146~22.04 \
intel-opencl-icd=25.18.33578.15-1146~22.04 \
libze-intel-gpu1 \
libze1 \
clinfo \
&& rm -rf /var/lib/apt/lists/*

# Synchronise le GID render avec l'hôte
ARG RENDER_GID=992
RUN groupmod -g ${RENDER_GID} render || groupadd -g ${RENDER_GID} render

##### ADD NEW DEPENDENCIES JUST ABOVE THIS LINE #####

```

FIGURE 9 – Ajout des bibliothèques dans le dockerfile

Maintenant dernière modification du Dockerfile pour python :

Ajout juste après « ADD NEW DEPENDENCIES ... » de :

```

# Installation Python et dpendances
RUN apt-get update && apt-get install -y \
libpython3-dev \
python3-pip \
python3-venv \
python3-catkin-pkg-modules

```



```
##### ADD NEW DEPENDENCIES JUST ABOVE THIS LINE #####
RUN apt-get update && apt-get install -y \
    libpython3-dev \
    python3-pip \
    python3-venv \
    python3-catkin-pkg-modules
RUN apt-get update && apt install --no-install-recommends -y libasio-dev
RUN apt-get update && apt-get install -y ros-humble-robot-state-publisher
RUN apt-get update && apt-get install -y ros-humble-joint-state-publisher-gui
RUN /usr/sbin/update-ccache-symlinks
ENV PATH="/usr/lib/ccache:${PATH}"
RUN apt-get update && apt-get install -y python3-colcon-common-extensions
RUN apt-get update && DEBIAN_FRONTEND=noninteractive apt install -y \
    ros-humble-pcl-ros \
    build-essential \
    libeigen3-dev \
    libjsoncpp-dev \
```

FIGURE 10 – Ajout des dépendances python dans le dockerfile

3.4.4 Modifications finales du build_and_run.sh

On va sourcer les dossiers des caches qu'on a créés pour que le Docker se serve des bibliothèques qu'il y a dedans. Comme ça, à chaque fois qu'on rebuild un Docker, on n'a pas besoin de tout retélécharger, c'est stocké dans les dossiers pour toujours.

Il faut ajouter les lignes suivantes dans le `docker_run` (les mettre au bon emplacement, avec les autres `-v`) :

```
-v $HOME/ros2-libs/pip-cache:/home/$USER/.cache/pip \
-v $HOME/ros2-libs/apt-cache:/var/cache/apt/archives \
-v $HOME/ros2-libs/python-packages:/home/$USER/.local/lib/python3.10/site-packages \
```

Enfin, dernière vérification : il faut mettre le bon display donc toujours dans le `docker_run`, changez `unix` par `:1` car c'est le bon display que `rviz2` sera capable de voir.

```
-e DISPLAY=unix:1 \
```

```

docker run \
  --net=host \
  --ipc=host \
  --pid=host \
  --restart=always \
  --init \
  --ulimit core=-1 \
  --privileged \
  -v /tmp/.X11-unix:/tmp/.X11-unix \
  -e DISPLAY=:1$DISPLAY \
  -v /dev:/dev \
  --group-add 992 \
  --env-file ./config/env_var.config \
  -it \
  --name $CONTAINER_NAME \
  -h $CONTAINER_NAME \
  --add-host $CONTAINER_NAME:"127.0.0.1" \
  -v $HOME/.ccache:$HOME/.ccache \
  -v $HOME/dataset_bdd100k:/home/user/dataset_bdd100k \
  -v $HOME/ros2-libs/pip-cache:/home/$USER/.cache/pip \
  -v $HOME/ros2-libs/apt-cache:/var/cache/apt/archives \
  -v $HOME/ros2-libs/python-packages:/home/$USER/.local/lib/python3.10/site-packages \
  -v $ABSOLUTE_WORKSPACE:$ABSOLUTE_WORKSPACE \
  -e ABSOLUTE_WORKSPACE=$ABSOLUTE_WORKSPACE \
  -e ROS_DOMAIN_ID=$ROBOT_ID \
  -w $ABSOLUTE_WORKSPACE \
  $IMAGE_NAME

```

FIGURE 11 – Modifications du build_and_run.sh

3.5 Build et lancement du Docker

Pour build et lancer, il faut exécuter le fichier `build_and_run.sh`, il faut donc aller dans le dossier dans lequel il est et faire les commandes suivantes (hors du docker) :

```

cd ~/roskit_ws/src/gr_docker
./build_and_run.sh

```

Normalement à cette étape, le Docker a été créé et on est à l'intérieur.

Mais si vous quittez le Docker en écrivant `exit`, vous pouvez rentrer à nouveau avec la commande :

```

docker exec -it gr-p247-humble-container bash

```

Si la création échoue allez voir dans les problèmes et solutions en annexe.

3.6 Installation des bibliothèques Python

Puisque nous venons de créer le Docker, il nous faut installer les bibliothèques car c'est la première fois. Une fois installées, elles seront stockées dans les dossiers créés précédemment et on n'aura plus besoin de le faire à nouveau.

Dans le terminal de l'hôte :

```

# Donne les bonnes permissions ton user
sudo chown -R $USER:$USER $HOME/ros2-libs/

```

Dans le terminal du Docker, il faut rentrer les lignes :

```

# Corrige les permissions sur .local
sudo chown -R user:user /home/user/.local/

# Installation des bibliothèques

```

```
pip install opencv-python ultralytics
```

3.7 Vérification du GPU

Pour vérifier que le GPU est détecté, on crée un fichier `test_intel.py` dans le `roskit_ws` et on le lance avec :

```
python3 test_intel.py
```

Contenu de `test_intel.py` :

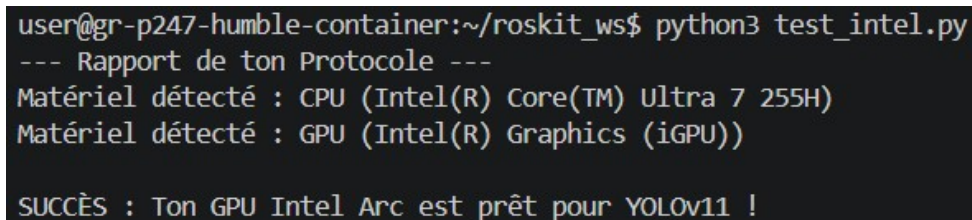
```
import opencv as cv

core = cv.Core()
devices = core.available_devices

print( --- Rapport de ton Protocole --- )
for device in devices:
    full_name = core.get_property(device, FULL_DEVICE_NAME )
    print(f Matériel détecté : {device} ({full_name}) )

if 'GPU' in devices:
    print( \nSUCCÈS : Ton GPU Intel Arc est prêt pour YOLOv11 ! )
else:
    print( \nATTENTION : Seul le CPU est visible. Vérifie tes accès Docker. )
```

On a fini de tout mettre en place ! On peut vérifier en lançant le programme `test_intel.py` dans le Docker : YOLOv11n est prêt à être utilisé !



```
user@gr-p247-humble-container:~/roskit_ws$ python3 test_intel.py
--- Rapport de ton Protocole ---
Matériel détecté : CPU (Intel(R) Core(TM) Ultra 7 255H)
Matériel détecté : GPU (Intel(R) Graphics (iGPU))

SUCCÈS : Ton GPU Intel Arc est prêt pour YOLOv11 !
```

FIGURE 12 – `test_intel.py` qui détecte le GPU

4 Partie 2 : Environnement ROS2

Pour cette partie, on va mettre en place l'environnement ROS2, c'est-à-dire création des packages, des codes, dépendances, etc.

4.1 Création du package

On va créer un package de type `ament_python` car c'est plus facile pour l'importation de YOLOv11n, modèle d'IA qu'on utilisera ici (à faire dans le `src/` du `ws`, dans le `docker`) :

```
ros2 pkg create --build-type ament_python detection_pkg \
  --dependencies rclpy sensor_msgs cv_bridge
```

On a donc un package nommé `detection_pkg` dans le Docker, on y placera les codes Python de détection.

4.2 Export du modèle YOLO en format OpenVINO

On va exporter YOLOv11 en format OpenVINO, donc pour l'optimiser pour le GPU Intel. Dans le terminal du Docker, mais avant on crée un dossier dans lequel on va mettre les modèles exportés en format OpenVINO :

```
cd ~/roskit_ws/src/detection_pkg
mkdir models
cd models
```

On est dans le bon dossier maintenant et donc on peut exporter le modèle :

```
python3 -c
from ultralytics import YOLO
model = YOLO('yolo11n.pt')
model.export(format='openvino', device='CPU')
print('Modele exporte dans yolo11n_openvino_model/')
```

4.3 Création du code de détection

Maintenant il faut ajouter le code Python de détection : on fait premièrement de la détection 2D avec YOLO. Créer le fichier `detector_2d.py` dans

`roskit_ws/src/detection_pkg/detection_pkg`.

`detector_2d.py` est fourni, vous pouvez le copier coller ou le télécharger.

Ce code utilise bien OpenVINO et le GPU intégré d'Intel.

4.4 Modification du `setup.py`

Il nous faut modifier le `setup.py` (qui est dans `roskit_ws/src/detection_pkg`) et y mettre celui fourni.

4.5 Modification du `package.xml`

Enfin, il faut rajouter des dépendances dans le `package.xml` (prendre celui fourni)

4.6 Compilation et lancement

Tous les codes et dépendances ont été mis en place, passons à la procédure pour lancer le code et les nœuds :

On se met dans le Docker, on rappelle qu'il faut toujours sourcer ROS au lancement :

```
source /opt/ros/humble/setup.bash
```

Maintenant on peut utiliser la commande `cm` qui permet de `colcon build` tous les packages du robot (`cm` est un alias, on peut retrouver sa signification exacte dans le `bashrc`).

Si lors de la compilation ça ne marche pas à cause de `numpy`, il faut le downgrader avec la commande :

```
pip install numpy<2
```

Si ça ne marche pas, ça peut aussi être dû à un problème autre comme par exemple vous qui avez mal fait la procédure, ou alors qu'il y a des problèmes de bibliothèques (voir problèmes en annexe pour la solution).

On peut maintenant lancer le nœud détection :

```
source $ABSOLUTE_WORKSPACE/install/setup.bash
ros2 run detection_pkg detector_2d --ros-args -p device:=GPU
```

Le nœud qu'on a créé avec notre code s'abonne au topic de la caméra et lance la détection.

Lancer noVNC sur Internet, se connecter au Docker. Si rviz2 est déjà lancé, vous pouvez cliquer en bas à gauche add -> by topic -> detection/image/image et la fenêtre avec la détection se lance !

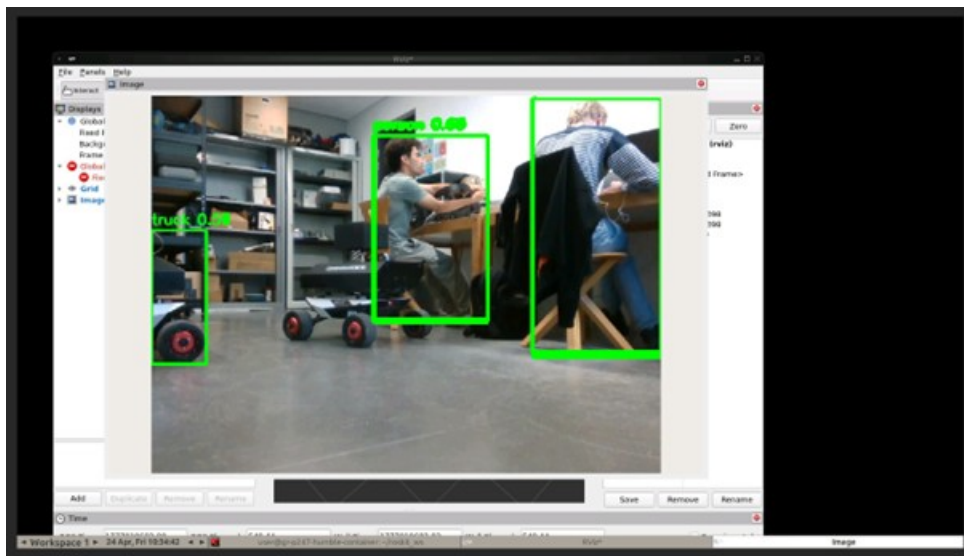


FIGURE 13 – Détection 2D avec Yolo

5 Annexe : Problèmes et solutions

- Si pendant le lancement du `build_and_run.sh` avec la commande `./build_and_run.sh` dans l'hôte, dans le bon dossier, vous avez une erreur avec `xhost`, il faut rajouter `|| true` après le `xhost` du `build_and_run.sh`.
- Si problème lors du lancement du nœud de détection de la caméra, voir la procédure de lancement.
- Si problème de version numpy, il faut downgrader sa version `< 2` car `cv_bridge` est compilé avec NumPy 1.x.
- Si le terminal de l'hôte n'a plus de couleur et que l'auto-complétion du Docker ne marche plus, c'est un léger problème de compatibilité avec les drivers Intel de PPA, il suffit juste d'écrire la commande à la main :

```
docker exec -it gr-p247-humble-container bash
```

- Si problème de display lors du lancement de `rviz2`, voir procédure de lancement. Si toujours un problème, la commande `export DISPLAY=:1` permet de connecter le bon display au conteneur.

```

11 class Detector2D(Node):
12     def __init__(self):
13         super().__init__('detector_2d')
14
15         package_share_dir = get_package_share_directory('detection_pkg')
16
17         self.declare_parameter('device', 'GPU')
18         self.declare_parameter('confidence', 0.5)
19         default_model_path = os.path.join(package_share_dir, 'models', 'yolo11n_openvino_model')
20         self.declare_parameter('model_path', default_model_path)
21
22         device = self.get_parameter('device').value
23         confidence = self.get_parameter('confidence').value
24         model_path = self.get_parameter('model_path').value

```

FIGURE 14 – Correction detector_2d.py

- C'est possible qu'avec `./build_and_run.sh` il y ait une erreur `curl: (6) Could not resolve host: repositories.intel.com`. Pour remédier au problème qui semble être dû au DNS, il faut modifier le `docker build` dans le `Dockerfile` et rajouter ces deux lignes au début :

```

--network=host \
--no-cache \

```

Le `no-cache` est juste pour maintenant, il faut le retirer quand le build a fonctionné.

- Il peut y avoir une erreur avec le code `detector_2d.py`, c'est sûrement dû au chemin avec le modèle, il faut rajouter :

```

from ament_index_python.packages import get_package_share_directory

```

- Si vous avez un problème de compilation du Docker, vous pouvez essayer de réinstaller certaines extensions et bibliothèques :

```

# Installation des outils
sudo apt update
sudo apt install -y \
    python3-colcon-common-extensions \
    python3-rosdep
cd ~/roskit_ws
sudo rosdep init
rosdep update
rosdep install --from-paths src --ignore-src -r -y

```

Puis :

```

cm

```

Il est possible aussi que ça fasse crasher le robot, donc à la place de `cm`, il faut faire :

```

colcon build --symlink-install \
    --cmake-args -DCMAKE_BUILD_TYPE=Release \
    --parallel-workers 1 # compile un seul la fois

```

6 Liste des commandes utiles

- Avec la commande `cat /etc/group`, on peut voir tous les groupes du système.
- Avec la commande `stat /dev/dri/renderD128`, on peut voir plus en détail ce groupe.
- `clinfo | grep "Device Name"` permet de voir les devices connectés à OpenCL.
- `echo $DISPLAY` donne la liste des displays, tu verras normalement rien, `unix` ou `:1`.